

American International University-Bangladesh
Department of Computer Science

CSC4164

Advanced Programming

.NET / ASP.NET Core MVC

Lab 4

Database-First EF Core

Course Code	CSC4164
Course Title	Advanced Programming (.NET / ASP.NET Core MVC)
Lab Session	Lab 4 — Week 4
Instructor	Jahid Hassan
Duration	2 Hours 20 Minutes (140 minutes)
Work Mode	Individual
Prerequisite	Labs 2 & 3 completed — routing, views, validation all working

1. Lab Objectives

By the end of this lab session, you will be able to:

- Create a SQL Server database and tables visually using SSMS
- Configure a database connection string in appsettings.json
- Scaffold an existing database into an ASP.NET Core MVC project using the EF Core CLI
- Understand the generated model classes and DbContext
- Register DbContext using dependency injection in Program.cs
- Perform a basic Read operation using EF Core

2. Prerequisites

- Labs 2 and 3 completed — routing, views, model binding, and validation all working
- SQL Server or SQL Server Express installed
- SQL Server Management Studio (SSMS) available
- Theory class Week 4 attended — Module 6 (Database-First EF Core)

3. Theory Recap

3.1 What is Database-First EF Core?

In the Database-First approach, the database already exists. You run a scaffold command that reads the database schema and automatically generates C# model classes and a DbContext class. You do not write these classes manually.

Component	What It Is	Where It Goes
Model class	C# class with properties matching each table column	Data/Entities/ folder
DbContext	Main class for querying and saving data — has a DbSet for each table	Data/ folder
DbSet<T>	Represents one table — used for all CRUD operations	Inside DbContext

3.2 The Scaffold Command

After adding your connection string to appsettings.json, you run one command in the terminal to generate everything:

```
Scaffold-DbContext "Name=ExampleDbContext" Microsoft.EntityFrameworkCore.SqlServer
-ContextDir Data -OutputDir Data/Entities
```

Part	Meaning
"Name=ExampleDbContext"	Read the connection string named ExampleDbContext from appsettings.development.json
Microsoft.EntityFrameworkCore.SqlServer	Use the SQL Server provider
-OutputDir Data/Entities	Put generated model classes in Data/Entities/
-ContextDir Data	Put the generated DbContext in Data/

3.3 Registering DbContext in Program.cs

After scaffolding, register the DbContext with the dependency injection container so controllers can use it:

```
// In Program.cs – before var app = builder.Build()
builder.Services.AddDbContext<YourContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("ExampleDbContext")));
```

Warning: After registering in Program.cs, delete the OnConfiguring method from the generated DbContext class. Otherwise the hardcoded connection string inside it will override your appsettings.json.

4. Lab Activities

Warning: Important: Type all code manually. Do not copy and paste from this manual.

★ Lab Task 1 — 10 Marks — 60 minutes

Task 1 Create the Database Visually in SSMS (15 min)

You will create a new database with a Department table using the visual table designer in SQL Server Management Studio.

Step 1 — Create a new database

1. Open SSMS and connect to your local SQL Server instance
2. Right-click Databases in Object Explorer → New Database
3. Name it SchoolDB and click OK

Step 2 — Create the Department table visually

4. Expand SchoolDB → Tables → right-click → New → Table
5. Add the following columns in the table designer:

Column Name	Data Type	Allow Nulls	Notes
DepartmentId	int	No	Right-click → Set Primary Key, then set Identity to Yes in Column Properties
DepartmentName	nvarchar(100)	No	
Location	nvarchar(100)	Yes	

6. Press Ctrl+S to save — name the table: Department
7. Right-click the Department table → Edit Top 200 Rows
8. Add 3 sample rows: Computer Science, Business Administration, Electrical Engineering

Task 2 Install Packages & Configure Connection String (20 min)

Step 1 — Install NuGet packages

9. Go to Tools → NuGet Package Manager → Manage NuGet Packages for Solution
10. Install the following three packages into FirstMVCApp:

Package	Purpose
Microsoft.EntityFrameworkCore.SqlServer	SQL Server provider for EF Core
Microsoft.EntityFrameworkCore.Tools	Enables EF Core CLI commands
Microsoft.EntityFrameworkCore.Design	Required for design-time scaffolding
Microsoft.EntityFrameworkCore	

Step 2 — Add connection string to appsettings.json

Open appsettings.development.json and add the ConnectionStrings section. Use ExampleDbContext as the key name:

```
{
  "ConnectionStrings": {
    "SchoolContext": "Data Source=DESKTOP-2C08H98;Initial Catalog=SchoolDB;
TrustServerCertificate=True;Integrated Security=True;"
  },
  "Logging": { ... }
}
```

Tip: If you are using SQL Server Express, change Server=. to Server=.\SQLEXPRESS in the connection string.

Task 3 Scaffold the Database & Register DbContext (20 min)

Step 1 — Run the scaffold command

11. Go to Tools → Nuget Package Manager → Package Manager Console
12. Confirm the prompt is in your project folder (FirstMVCApp directory)
13. Type and run the following command:

```
Scaffold-DbContext "Name= SchoolContext" Microsoft.EntityFrameworkCore.SqlServer -
ContextDir Data -OutputDir Data/Entities
```

Step 2 — Review generated files

14. Expand Data/Entities — confirm Department.cs is generated
15. Expand Data/ — confirm SchoolContext.cs is generated
16. Open Department.cs — read the generated properties
17. Open SchoolContext.cs — find the DbSet<Department> property and the OnConfiguring method

Step 3 — Register DbContext in Program.cs

18. Open Program.cs and add the following before var app = builder.Build():

```
builder.Services.AddDbContext<SchoolContext>(options =>
options.UseSqlServer(builder.Configuration.GetConnectionString("SchoolContext")));
```

Task 4 Read Data from the Database (25 min)

You will create a DepartmentController and an Index view that reads and displays all departments from the database.

Step 1 — Create DepartmentController

Create a new empty MVC controller named DepartmentController. Inject the SchoolContext via constructor and implement the Index action:

```
private readonly SchoolContext _context;

public DepartmentController(SchoolContext context)
{
    _context = context;
}

public IActionResult Index()
{
    var departments = _context.Departments.ToList();
    return View(departments);
}
```

Step 2 — Create the Index view

Create Views/Department/Index.cshtml. Use plain HTML for the table structure and Razor only for dynamic data:

```
@model List<FirstMVCApp.EF.Tables.Department>

@{ ViewData["Title"] = "Departments"; }

<h2>Departments</h2>

<table border="1" cellpadding="6" cellspacing="0">
    <thead>
        <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Location</th>
        </tr>
    </thead>
    <tbody>
        @foreach (var dept in Model)
        {
            <tr>
                <td>@dept.DepartmentId</td>
                <td>@dept.DepartmentName</td>
                <td>@dept.Location</td>
            </tr>
        }
    </tbody>
</table>
```

19. Run the application and navigate to /Department/Index

20. Confirm all 3 departments appear from the database

Tip: If you see zero rows but no error, check that you actually inserted sample data in SSMS. If you see a connection error, double-check your connection string in appsettings.json.

5. Common Errors and Solutions

Error	Likely Cause	Solution
Scaffold command fails	Wrong server name in connection string	Try Server=.\SQLEXPRESS or check your SQL Server instance name in SSMS
TrustServerCertificate error	SSL not trusted on local dev machine	Add TrustServerCertificate=True to connection string in appsettings.json
No service registered for SchoolContext	DbContext not added in Program.cs	Add <code>builder.Services.AddDbContext<SchoolContext>(...)</code> before <code>builder.Build()</code>
Zero rows returned from database	Sample data not inserted in SSMS	Right-click the table in SSMS → Edit Top 200 Rows and add data

6. Completion Checklist

#	Checklist Item	Done
1	SchoolDB created with Department table and 3 sample rows in SSMS	☐
2	EF Core packages installed successfully	☐
3	Connection string added to appsettings.development.json with key SchoolContext	☐
4	Scaffold command executed — Department.cs and SchoolContext.cs generated in Data/ folders	☐
5	DbContext registered in Program.cs	☐
7	DepartmentController Index action reads from DB and view displays all departments	☐

7. Self-Assessment Questions

Review these questions before leaving. They may appear in Quiz 1.

- What does the `dotnet ef dbcontext scaffold` command do?
- What is a `DbSet<T>` and what does it represent?
- Why do we remove the `OnConfiguring` method after registering `DbContext` in `Program.cs`?
- What is the difference between Database-First and Code-First EF Core?
- Why does an in-memory list reset on every request if not declared as static?

8. Further Reading

- EF Core scaffolding: <https://learn.microsoft.com/ef/core/managing-schemas/scaffolding>
- DbContext configuration: <https://learn.microsoft.com/ef/core/dbcontext-configuration>

End of Lab 4 — Lab 5 covers Code-First EF Core.